

FPGA Electromagnetic Field Renderer

Project brief & implementation plan — PYNQ-Z1 target

Concept. A real-time visualisation of an electromagnetic wave field computed analytically per pixel from one or more antenna sources. The inner loop is offloaded to a custom SystemVerilog accelerator on the PYNQ-Z1's PL fabric, the ARM PS orchestrates frames and the display, and a browser/tablet UI lets users place antennas and sweep parameters live. The brief below corrects the source plan's spatial-parallelism strategy to fit the Zynq-7020's resources and adds the missing UI / number-format / benchmarking work needed to meet every numbered requirement.

1. Requirements compliance review

Each numbered requirement is reviewed against the source brief. The physics and parallelism story is solid; the gaps are in hardware sizing, UI, and verification.

Req	Statement (abridged)	Source brief	Action in this plan
1.1	Computationally intensive function	✓ Per-pixel cos, $1/\sqrt{r}$, atan2 over N sources	Keep — measure cost in §3
1.2	Embarrassingly parallel	✓ No neighbour deps; superposition only	Keep — exploit at tile level (§4)
2.1	SystemVerilog accelerator	Implied	Defined as PE + sequencer (§4–5)
2.2	CPU↔FPGA parameter interface	Implied	AXI-Lite register map (§5.3)
2.3	Number-format / word-length trade-off	✗ Not addressed	Fixed-point analysis (§6)
2.4	Beat a CPU-only C/C++/Cython baseline	✗ No baseline plan	Cython reference + benchmark (§7)
3.1	UI may be on separate hardware	✗ Not addressed	Tablet/browser over WiFi (§8)
3.2	Intuitive parameter adjustment	✗ Not addressed	Drag antennas, slider sweeps (§8)
3.3	Informational overlay	✗ Not addressed	HUD with f, λ , mode, fps (§8)

Verdict. The source brief fulfils Requirement 1 outright. Requirement 2 is partially specified: the accelerator concept is right but the hardware sizing assumes 40,000 fully-parallel PEs, which the Zynq-7020 cannot host (≈ 220 DSP48 slices, ≈ 13.3 k logic slices). Requirement 3 is absent. Sections 4–8 below address all three.

2. The physics being rendered

The model is the analytic far-field of a set of phased point (or patterned) sources. For source n at position $\mathbf{r}_n$ with amplitude A_n , phase ϕ_n , radiation pattern $D_n(\theta)$ and free-space wavenumber $k = 2\pi/\lambda$, the instantaneous electric field contribution at a grid point $\mathbf{r}$ is:

$$E_n(\mathbf{r}, t) = A_n \cdot D_n(\theta_n) \cdot (1/r_n) \cdot \cos(\omega t - k r_n + \phi_n)$$

with $r_n = |\mathbf{r} - \mathbf{r}_n|$ (Euclidean distance). The displayed field is the linear superposition over all sources:

$$E(\mathbf{r}, t) = \sum_n E_n(\mathbf{r}, t)$$

Why this is FPGA-friendly. Each pixel evaluation is independent (no neighbour reads), uses a fixed set of trig and reciprocal-square-root operations, and depends on a small broadcast parameter set (the antenna list). There is no PDE update, no boundary condition, no CFL stability criterion — the wave is *evaluated* at points rather than *evolved* through them. That replaces a time-stepped stencil with a pipelined arithmetic dataflow, which is exactly what FPGA fabric is good at.

What it isn't. This is not a Maxwell solver. Near-field structure (reactive $1/r^2$ and $1/r^3$ terms), multipath off scatterers, and resonant cavities are out of scope. The model is appropriate for the educational target — interference, beam-forming, and channel intuition — and is extendable later (see §10).

Example fields the renderer will produce

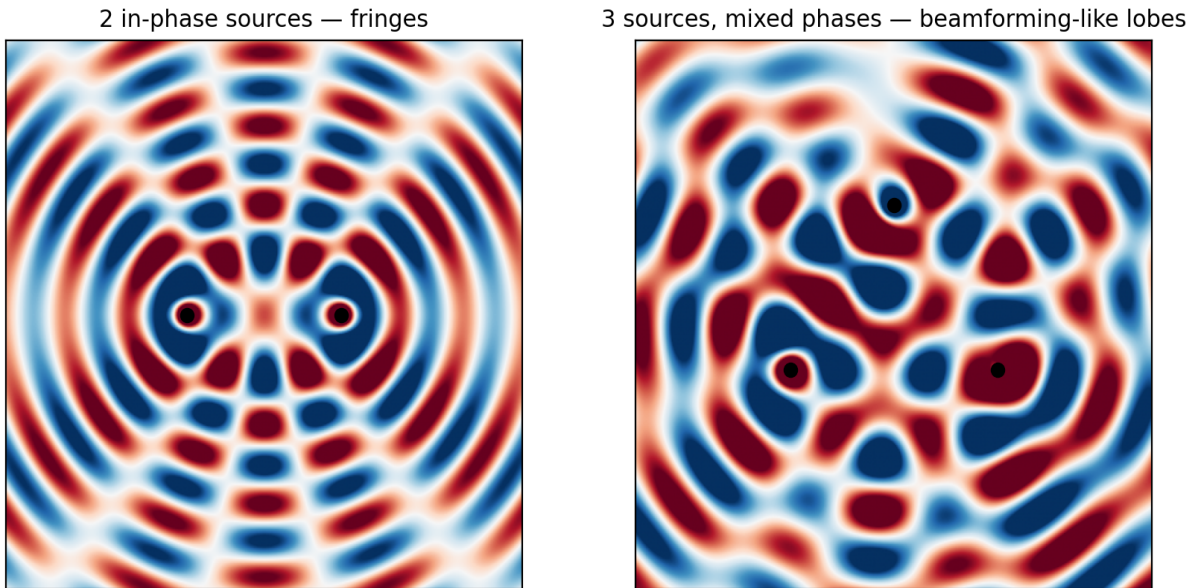


Figure 1 — Two reference outputs from the same analytic model used as ground truth for verifying the FPGA accelerator.
Left: classic two-slit-style fringes. Right: three sources with mixed phases showing beam-forming-like lobes.

3. CPU baseline (the bar to beat)

Requirement 2.4 demands the accelerator outperform a CPU-only implementation. To make that claim defensible, the same physics model is implemented twice on the Cortex-A9 PS before any RTL is written:

- **NumPy reference** — readable, vectorised, used as the bit-accuracy golden model for the FPGA. Slow ($\approx 1\text{--}4$ fps at 200×200 , 5 sources) but correct.
- **Cython / C extension** — same model rewritten with typed loops, OpenMP across the dual A9 cores, and a single-precision math path. This is the honest CPU baseline that the FPGA must beat.

Both will be profiled with `perf` to get cycles-per-pixel numbers. The Cython version's throughput is the headline figure the FPGA accelerator is benchmarked against in §11.

4. System architecture

The system splits naturally along the Zynq-7020 PS/PL boundary. The PS handles soft state (antenna list, mode flags, frame timing, display driving, UI server). The PL holds the pipelined arithmetic dataflow and feeds a DMA stream into a DDR frame buffer. The display and UI both live on external hardware reached over standard interfaces.

Top-level system architecture

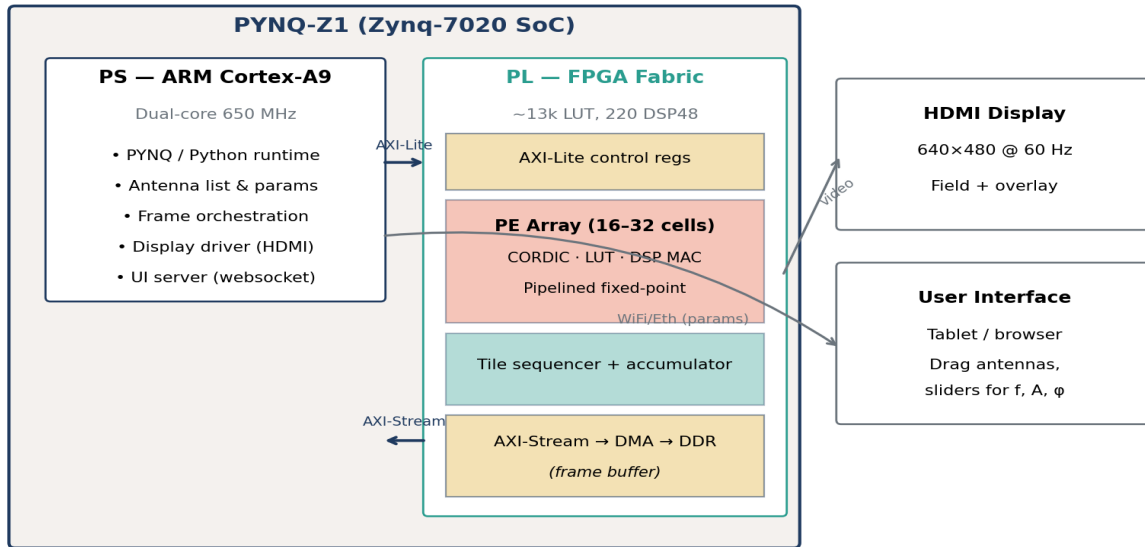


Figure 2 — Top-level system architecture. The PS configures the PL through AXI-Lite registers and reads completed frames from DDR over AXI-Stream / DMA. Display and UI are external.

4.1 Why not one PE per pixel?

The source brief proposes a literal 200×200 PE grid. The Zynq-7020 offers roughly 220 DSP48E1 slices and 13.3 k logic slices. A single PE needs at minimum two DSPs (for the squared-distance multiplies and the final $A \cdot D \cdot (1/r) \cdot \cos$ product), so the hard ceiling is on the order of **~100 PEs**, not 40,000. The realistic operating point is 16–32 PEs in parallel, with the grid swept by tile streaming.

4.2 Tile-streamed parallelism

The 200×200 output grid is partitioned into tiles (e.g. 8×8 = 64 pixels per tile, 625 tiles per frame). The PE array evaluates one tile per pass; the tile sequencer in the PL walks raster order over the grid and the DMA engine streams each completed tile into the DDR frame buffer. Because each pixel is independent, tile order and tile size are pure performance knobs — they don't affect correctness.

Tile-streamed parallelism — small PE array covers a large grid

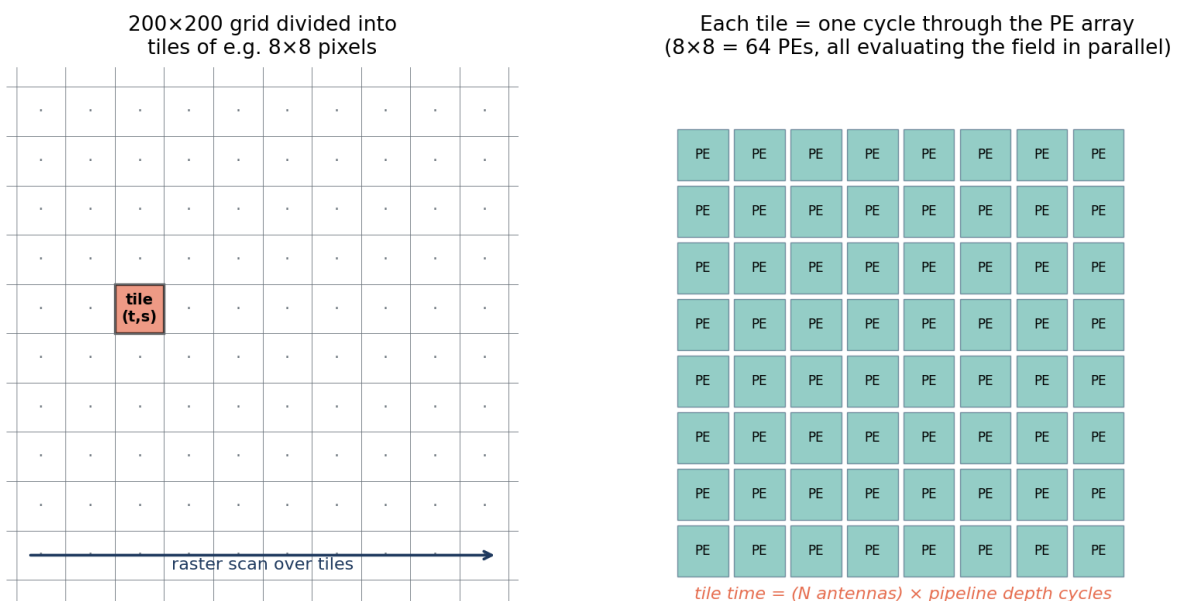


Figure 3 — Tile-streamed parallelism. Left: the 200×200 grid is divided into tiles processed in raster order. Right: each tile is evaluated in parallel by the PE array (illustrated as 8×8 = 64 PEs).

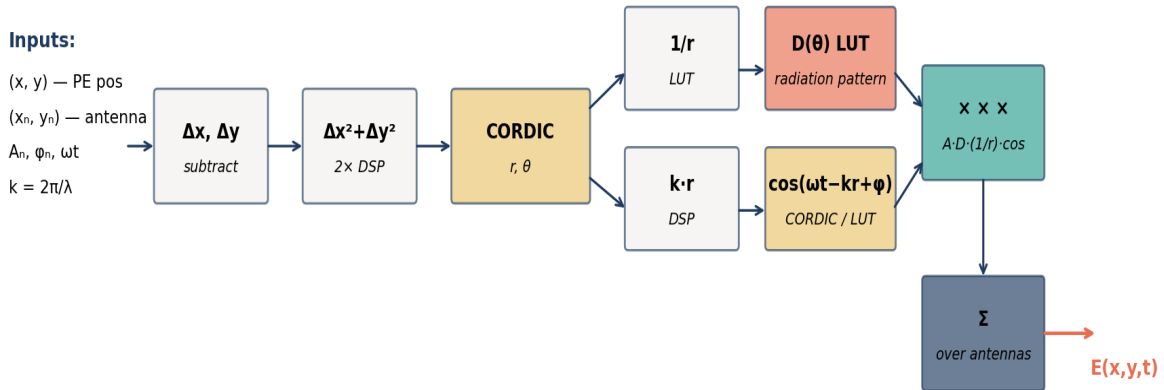
5. The accelerator — datapath, control, interface

5.1 Per-PE datapath

One PE evaluates one antenna's contribution per clock once its pipeline is filled, and accumulates over the antenna list. The arithmetic chain is:

- **Geometry.** Δx , Δy via subtractors; r^2 via two DSP multiplies; (r, θ) via a vectoring-mode CORDIC (gives $\sqrt{}$ and atan2 from the same core).
- **Propagation.** $k \cdot r$ via one DSP multiply; $\cos(\omega t - k r + \phi)$ via a rotation-mode CORDIC or a sin/cos LUT addressed by a quantised phase. LUT is smaller; CORDIC is more flexible for arbitrary phase resolution.
- **Pattern.** $D(\theta)$ via a small LUT indexed by θ from the same CORDIC. Isotropic = constant; $\sin(\theta)$ dipole = one extra LUT.
- **Amplitude.** $1/r$ via a LUT (input r from CORDIC, modest precision is fine because the visualisation tone-maps anyway).
- **Combine.** One DSP multiply for $A \cdot D \cdot (1/r) \cdot \cos$, then accumulate.

Per-PE datapath — one antenna contribution, accumulated over N



Pipelined fixed-point datapath. Suggested formats: Q4.12 for r , Q1.15 for $\cos/\sin$, Q2.14 for accumulator (with guard bits).

Stages registered → one new pixel result per clock once filled.

Figure 4 — Pipelined per-PE datapath. Pale boxes are arithmetic, gold are CORDIC, orange is the radiation-pattern LUT, teal is the final multiplier chain, dark is the antenna-loop accumulator.

5.2 Control: tile sequencer

A small finite-state machine in the PL drives the array: it broadcasts the current antenna parameters (one antenna per cycle, looped through the active list), strobes each PE's pixel coordinate (tile origin + PE index), waits for pipeline drain, and signals tile-complete to the DMA. All antennas are kept in a tiny on-chip BRAM written by the PS over AXI-Lite — typically a few dozen entries, well under 1 KB.

5.3 PS↔PL interface (AXI-Lite register map)

Offset	Reg	Direction	Purpose
0x00	CTRL	PS→PL	start, reset, mode bits
0x04	STATUS	PL→PS	busy, frame_done, error
0x08	N_ANTENNAS	PS→PL	active source count
0x0C	OMEGA_T	PS→PL	current ωt phase word

0x10	K_WAVE	PS→PL	wavenumber k
0x14	OUT_MODE	PS→PL	0=E, 1= E , 2= E ² , 3=φ
0x18	FRAME_BASE	PS→PL	DDR frame buffer address
0x100+	ANT_RAM[i]	PS→PL	x, y, A, φ, pattern_id per source

This map is exposed to the PS as a PYNQ MMIO region; the Python control code reads/writes registers directly. A frame is a sequence of: write antennas → set OMEGA_T → pulse CTRL.start → wait for STATUS.frame_done → display.

6. Number format and word-length analysis

Floating-point at every pipeline stage is wasteful on Zynq-7020; fixed-point with stage-specific widths is the standard answer. The choice is driven by the dynamic range each variable actually needs:

Variable	Range	Format	Justification
x, y, Δx, Δy	−256 ... +256	Q9.7 (16-bit signed)	Grid is 200×200; ±256 leaves headroom.
r ² (intermediate)	0 ... 2 ¹⁷	Q18.14 (32-bit)	Up to 2 × 256 ² before sqrt.
r (CORDIC out)	0 ... ~360	Q9.7 (16-bit)	√2·256 ≈ 362. Matches 1/r LUT input.
θ	−π ... +π	Q3.13 (16-bit signed)	Plenty of angular resolution for pattern LUT.
k·r (phase)	wraps mod 2π	Q3.13 (16-bit)	Only the fractional part feeds cos.
cos(·), sin(·)	−1 ... +1	Q1.15 (16-bit signed)	Standard DSP fixed-point trig.
1/r	~0.003 ... 1	Q1.15 (16-bit)	Saturate near r→0 to a clamp.
Field accumulator	−N ... +N	Q5.18 (24-bit signed)	Guard bits for up to 32 sources.

Verification of the choice. The NumPy reference is re-run in fixed-point emulation (using the same widths) and the resulting frame is compared against the float64 model. Acceptance criterion: PSNR > 40 dB on a representative test bench of source configurations. Bit widths are tightened until either PSNR or resource usage becomes the binding constraint, then traded.

7. Verification & benchmarking

- **Unit tests.** Each Verilog module (CORDIC, 1/r LUT, multiplier chain, accumulator) has a cocotb testbench driven by NumPy-generated stimulus and golden output.
- **Integration test.** A full tile of the accelerator is simulated against the float64 reference for a fixed antenna config; pass = PSNR > 40 dB.
- **On-hardware sanity.** A known interference pattern (two in-phase sources, see Figure 1 left) is rendered and compared visually and numerically to the reference.
- **Benchmark.** Frames-per-second measured for 200×200 output, 1 / 5 / 16 sources, and four output modes, against the Cython CPU baseline on the same A9 cores. Target: ≥ 10× throughput at 5 sources.

8. User interface and educational overlay

Requirement 3 is fulfilled by splitting the UI from the visualisation host (3.1). The visualisation FPGA renders to an HDMI display; control lives on a tablet/phone or laptop running a web app that talks to a small websocket server on the PS.

8.1 Control surface (Req 3.2)

- **Drag-to-place antennas** on a canvas matching the rendered grid. Each new antenna gets a colour and a label (A1, A2, ...).
- **Per-antenna sliders** for amplitude A, phase ϕ , and pattern (isotropic / dipole / array preset).
- **Global controls:** frequency f (and derived λ), time-step rate, pause/play, reset.
- **Preset scenarios** for teaching: two-slit fringes, phased-array beam steering, near-vs-far field contrast (when §10 is implemented).

8.2 Informational overlay (Req 3.3)

A heads-up display rendered by the PS onto the framebuffer (or composited at HDMI scan-out) shows:

- Current f , λ , ωt , output mode, source count.
- Live throughput (fps) and CPU-baseline ratio — the FPGA win, visible while you teach.
- Optional ruler in wavelengths, and ‘wavefront’ contour lines drawn when $|\cos(\omega t - k r + \phi)|$ crosses zero.

User interface concept — display + tablet control panel

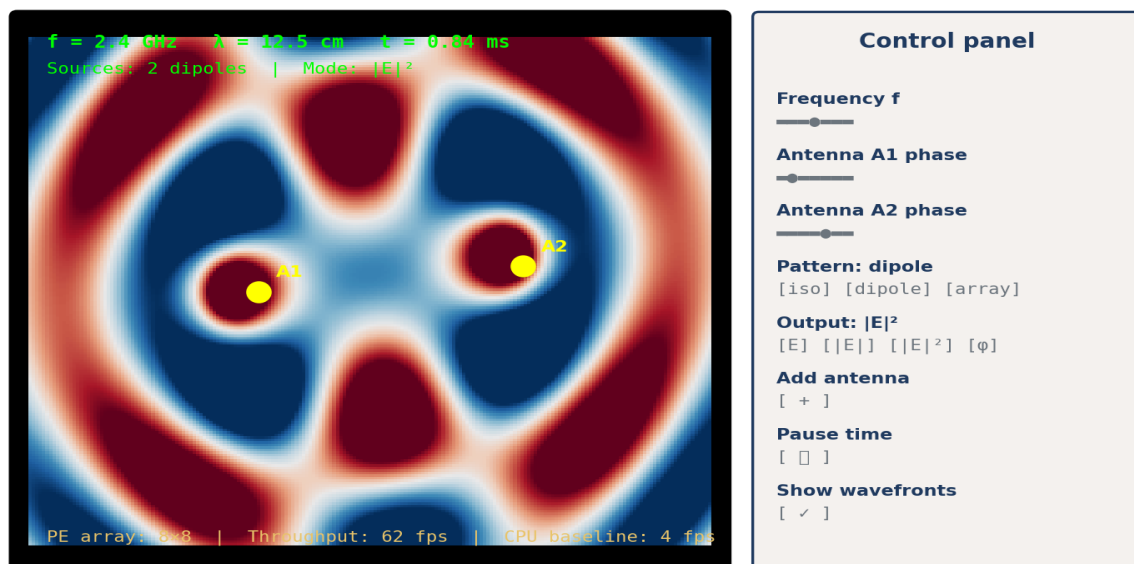


Figure 5 — UI concept. Left: HDMI output with HUD overlay and draggable antenna markers. Right: tablet/browser control panel served by the PS over WiFi.

9. Step-by-step implementation plan

Step 1 — Lock the spec and the golden model

Write the NumPy reference renderer (200×200, N sources, far-field model from §2). Generate a small library of canonical test scenes (single isotropic, two-source fringes, three-source beam, dipole pattern). This is the bit-accuracy oracle for everything that follows.

Step 2 — Build the CPU baseline

Port the renderer to Cython with OpenMP across both A9 cores, single-precision math, vectorised inner loops. Measure fps for each test scene. This is the bar the FPGA must beat (Req 2.4).

Step 3 — Fixed-point feasibility study

In NumPy, emulate the §6 fixed-point widths end-to-end. Confirm PSNR > 40 dB on every test scene. Adjust widths until tight.

Step 4 — Single PE in SystemVerilog

Implement one PE: subtract → square → CORDIC → $1/r$ LUT → $D(\theta)$ LUT → cos via second CORDIC → multiply → accumulate. cocotb tests against the fixed-point NumPy emulation. Pipeline registered between every operator; target Fmax ≥ 100 MHz.

Step 5 — PE array and tile sequencer

Replicate the PE 16–32× and wrap with the tile-walk FSM (§5.2). Simulate a complete tile against the reference. Confirm tile time = $N_{\text{antennas}} \times \text{pipeline_depth}$ cycles.

Step 6 — AXI integration and PYNQ overlay

Wrap the array as an IP with AXI-Lite control registers (§5.3) and an AXI-Stream master to DMA. Build the Vivado block design, generate the bitstream, package as a PYNQ overlay. Smoke-test from a Jupyter notebook on the board.

Step 7 — Display pipeline

Drive HDMI at 640×480/60 Hz from the DDR framebuffer. Up-scale the 200×200 field with nearest-neighbour or bilinear. Tone-map $E \rightarrow \text{colour}$ (diverging RdBu for signed, viridis for $|E|^2$).

Step 8 — UI server and overlay

Run a small Python websocket server on the PS. Browser app sends JSON antenna lists; server pushes them through AXI-Lite. PS renders the HUD onto the framebuffer in software (text is too low-bandwidth to deserve PL).

Step 9 — Benchmark and iterate

Run the §7 benchmark suite. If the FPGA loses on any scene, profile: is the bottleneck antenna loop length, tile overhead, or memory bandwidth? Tighten the right knob (more PEs, deeper pipeline, larger tiles).

Step 10 — Educational features and demo

Add preset scenarios, wavefront overlay, slow-motion mode, and a side-by-side ‘FPGA vs CPU’ view that runs both renderers and shows the live fps gap. This is the demo-day artefact.

Indicative 9-week implementation plan

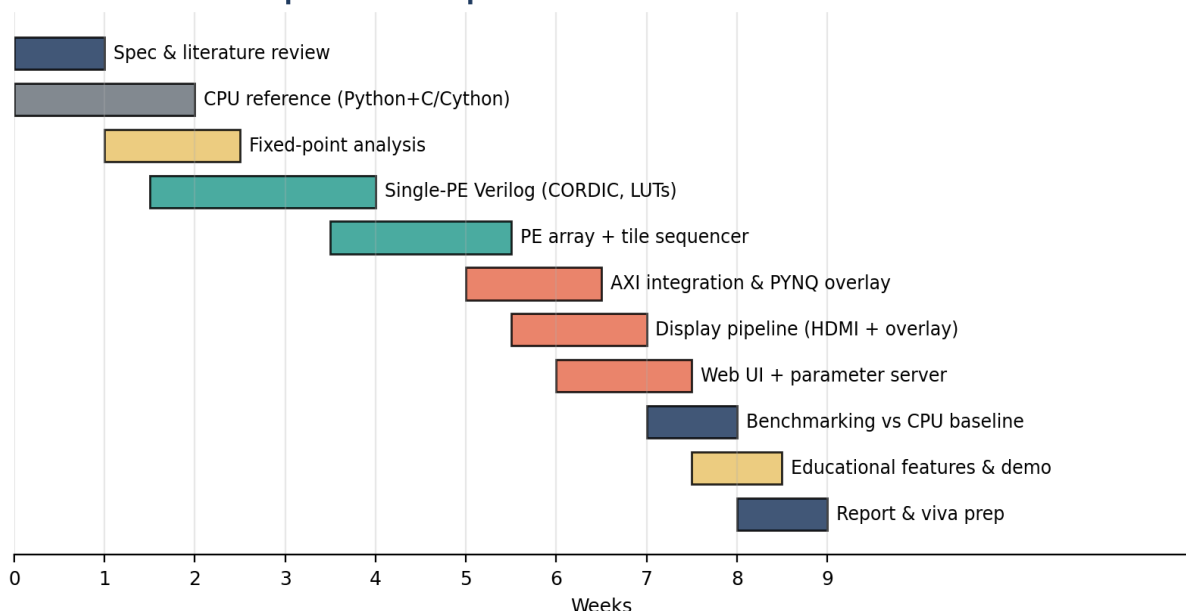


Figure 6 — Indicative 9-week schedule. Reference model and CPU baseline run early in parallel; the FPGA path (teal/orange) is the critical chain; UI and benchmarking pick up once a working overlay exists.

10. Extensions (if time allows)

- **Near-field terms.** Add the $1/r^2$ and $1/r^3$ contributions from the source brief §11. Cost: a couple of extra LUTs and one more multiply in the per-PE chain. Makes the educational story richer: students see the far-field approximation visibly break near a source.
- **Distributed antennas.** Represent a patch antenna as 8–16 elementary emitters with phase/amplitude tapers. No architectural change — just a longer antenna list. Lets you demonstrate beam steering and aperture effects.
- **Noise model.** Add an LFSR-based noise term to amplitude and phase. Trivially parallel; useful for the educational ‘SNR’ story.
- **Receiver probe.** Sample E at one point and plot the time series in the UI — turns the visualiser into a comms-channel demo.

11. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
DSP usage exceeds budget	Medium	High	Start with 16 PEs; profile early; share CORDIC across stages.
Timing closure below 100 MHz	Medium	Med	Register every operator output; pipeline CORDIC fully.
Fixed-point loses too much accuracy	Low	Med	Already de-risked by §3 study before RTL is written.
HDMI driver complexity	Med	Low	Use Diligent's reference HDMI IP from the PYNQ-Z1 base overlay.
UI latency feels sluggish	Low	Med	Apply parameters at frame boundaries only; cap UI rate to 30 Hz.
FPGA loses the benchmark	Low	High	Re-tune tile size, deepen pipeline, or accept fewer PEs at higher clock — but be honest in the report either way.

12. Summary

The source brief's algorithm is FPGA-perfect: a closed-form, neighbour-free, embarrassingly parallel field evaluation. The two things it doesn't say — how to fit 40,000 cells' worth of compute into a Zynq-7020, and how the human in front of the demo actually interacts with it — are exactly the contributions of this plan. A 16–32-PE pipelined array streamed over tiles, AXI-Lite parameter control, a Cython baseline to beat, fixed-point widths chosen by PSNR, and a browser-tablet UI with an informational overlay together satisfy every numbered requirement.

End of brief.